🛡️ CLOUDSENTINEL — ENGINEERING HANDBOOK

An Intelligent Cloud Security Posture Assessment (CSPM) & Threat Detection Platform for AWS Environments

Author: Ayush Kumar Patel

Repository: github.com/ayushkp930/CloudSentinel

Stack: Python 3, Streamlit, Boto3, SQLite, Pytest, Ruff

9 Rules	35 / 35	SHA-256	100% Safe
5 Posture + 4 SIEM Event Rules	Automated Pytest Suite Passing	Stateful Alert Deduplication	Offline-First & Read-Only AWS

01. Executive Summary & Core Problem Statement

Modern cloud infrastructures suffer from two primary operational blind spots: static resource misconfigurations (such as unrestricted SSH/RDP security groups, misconfigured S3 Public Access Blocks, and overly permissive wildcard IAM policies) and suspicious control-plane activities buried inside AWS CloudTrail audit logs (such as audit trail tampering, unauthorized privilege escalation, and credential brute-force attacks). Commercial Cloud Security Posture Management (CSPM) and SIEM platforms are often expensive, opaque in how they calculate priority scores, or risky to test against live environments.

CloudSentinel solves this by implementing an auditable, local-first, modular Python security platform. It operates out-of-the-box using deterministic synthetic fixtures without requiring an AWS account or incurring cloud costs, while providing an optional, strictly **read-only Boto3 collector** gated by explicit analyst confirmation.

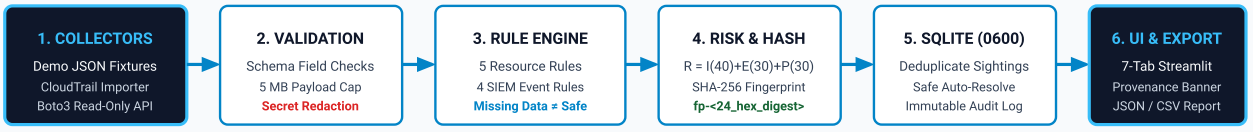


Figure 1.1: CloudSentinel 6-Stage Deterministic Processing Architecture (Collect → Validate → Detect → Score → Persist → Export)

💡 **Quick Interview Pitch (30-Second Summary for Viva & Recruiters):**
"CloudSentinel is a local-first Python CSPM and CloudTrail threat detection engine for AWS. Instead of relying on black-box alerts, it evaluates 9 deterministic security rules across EC2 Security Groups, S3 Buckets, IAM Policies, and CloudTrail logs. It computes an explainable heuristic risk score (*R = Impact + Exposure + Privilege*), prevents alert fatigue via canonical SHA-256 fingerprinting in SQLite, refuses to falsely resolve issues during failed or partial scans, and exposes a 7-tab Streamlit analyst workbench backed by 35 automated pytest tests."

02. Complete Codebase Directory & Module Responsibility Matrix

CloudSentinel follows a strict layered architecture where the Streamlit user interface (`app.py`) never executes raw detection logic or direct SQL queries. Every responsibility is isolated inside `src/cloudsentinel/` so that collectors, rules, scoring, database persistence, and exports can be unit-tested independently.

File Path	Architectural Layer	Primary Responsibility & Key Functions
`app.py`	Presentation (UI)	Streamlit 7-tab dashboard (`main()` `load_settings()`). Renders metrics, provenance banners, filters, status transitions, and download buttons.
`config/settings.yaml`	Configuration	Central non-secret YAML config defining fixture paths, SQLite path, heuristic score bounds (40/30/30), and `AUTH-001` brute-force thresholds.
`src/.../utils/logging_config.py`	Core Utilities	Configures structured UTC console logging (`get_logger()`) with zero secret disclosure.
`src/.../utils/validation.py`	Input Guardrails	Schema validators (`validate_resource_item()`, `validate_event_item()`) that reject malformed dicts or missing mandatory keys before rule evaluation.
`src/.../collectors/demo_provider.py`	Data Collection	Offline fixture loader (`load_demo_bundle()`). Reads `resources.json` and `cloudtrail_events.json` and capture corrupt JSON errors as diagnostics
`src/.../collectors/cloudtrail_importer.py`	SIEM Ingestion	Normalizes custom & native AWS CloudTrail `Records` (`normalize_raw_cloudtrail_record() enforces 5 MB cap, and scrubs secrets (`redact_sensitive_fields()`)
`src/.../collectors/aws_provider.py`	Cloud Integration	Strictly read-only Boto3 collector (`collect_aws_readonly_bundle()`, `_normalize_security_group()`). Captures `AccessDenied` as diagnostics (`TC-10`).
`src/.../rules/{base,network,s3,iam,event}_rules.py`	Detection Engine	Defines `DetectionRule` contract (`base.py`) and implements all 9 deterministic detectors (`NET-001..003`, `S3-001`, `IAM-001..002`, `LOG-001`, `AUTH-001`, `KEY-001`).
`src/.../engine/{scanner,risk_scoring,deduplication}.py`	Orchestration & Math	Coordinates scan execution (`run_resource_scan()`), computes heuristic priority (`calculate_heuristic_risk()`), and hashes findings (`compute_finding_fingerprint()`).
`src/.../db/{models,repository}.py & reports/`	Storage & Reporting	Initializes 4 SQLite tables with `0o600` permissions (`init_db()`), manages deduplicated CRUD (`CloudSentinelRepository`), and

04. Deep Dive: The 9 Deterministic Detection Rules (`rules/`)

Every resource rule inherits from `DetectionRule` (`src/cloudsentinel/rules/base.py`) and returns a `RuleEvaluationOutcome(rule_id, resource_id, matches, diagnostics)` . Every confirmed match emits a `RuleMatch` carrying structured JSON evidence and three heuristic factor weights (`impact_score` , `exposure_score` , `privilege_score`).

Rule ID	Class & Module	Deterministic Evaluation Logic & Edge Cases	Score (I+E+P)	Severity
`NET-001`	`PublicSSHRule` (`network_rules.py`)	Uses helper `_port_in_range(22, proto, from_p, to_p)` to check if TCP/22 or all-traffic (`-1`) is open to IPv4 `0.0.0.0/0` .	30 + 30 + 14 = **74**	High
`NET-002`	`PublicRDPRule` (`network_rules.py`)	Checks if Windows Remote Desktop port `TCP/3389` (or a port range covering 3389) allows unrestricted IPv4 `0.0.0.0/0` .	32 + 30 + 14 = **76**	High
`NET-003`	`PublicIPv6AdminPortRule` (`network_rules.py`)	Evaluates `ipv6_ranges` separately for `::/0` exposure across administrative ports `22` and `3389` .	30 + 30 + 14 = **74**	High
`S3-001`	`S3EffectivePublicAccessRule` (`s3_rules.py`)	Computes *effective* exposure: triggers only if ACL is in `PUBLIC_ACLS` while `ignore/block_public_acls` is False, OR policy is public while `restrict/block_public_policy` is False.	38 + 30 + 18 = **86**	Critical
`IAM-001`	`IAMBroadPermissionsRule` (`iam_rules.py`)	Inspects policy statements for `Effect == "Allow"` with wildcard action (`"*"` or `"iam:*"`), wildcard resource (`"*"`), and empty `conditions` .	35 + 15 + 30 = **80**	Critical
`LOG-001`	`CloudTrailTamperingRule` (`event_rules.py`)	Detects successful CloudTrail defense-evasion calls in `{"StopLogging", "DeleteTrail", "UpdateTrail"}` (MITRE T1562.008).	38 + 20 + 28 = **86**	Critical
`IAM-002`	`PrivilegedPolicyAttachmentRule` (`event_rules.py`)	Flags `AttachUserPolicy` / `AttachRolePolicy` / `PutUserPolicy` events attaching `AdministratorAccess` , `PowerUserAccess` , or `IAMFullAccess` .	32 + 16 + 28 = **76**	High
`AUTH-001`	`RepeatedFailedLoginRule` (`event_rules.py`)	Groups failed `ConsoleLogin` events by `actor` , sorts timestamps, and applies a 15-minute sliding window (threshold ≥ 3). Emits 1 aggregated alert per actor.	26 + 24 + 18 = **68**	High
`KEY-001`	`AccessKeyCreationRule` (`event_rules.py`)	Flags successful `CreateAccessKey` events as persistence indicators requiring analyst verification with the identity owner.	20 + 15 + 20 = **55**	Medium

05. Crucial Rule Algorithms Explained (For Technical Interviews)

- **Why `_port_in_range()` in `network_rules.py` is smarter than a simple `port == 22` check:** In AWS EC2 Security Groups, an administrator might open a range such as `from_port: 0, to_port: 1024` or allow all protocols (`protocol: "-1"`). A naive check for `from_port == 22` would miss this completely! Our helper

06. Heuristic Risk Engine, SHA-256 Fingerprinting & SQLite Lifecycle

Once rules emit matches, the engine layer (`src/cloudsentinel/engine/`) and persistence layer (`src/cloudsentinel/db/`) transform raw matches into prioritized, deduplicated, stateful security findings.

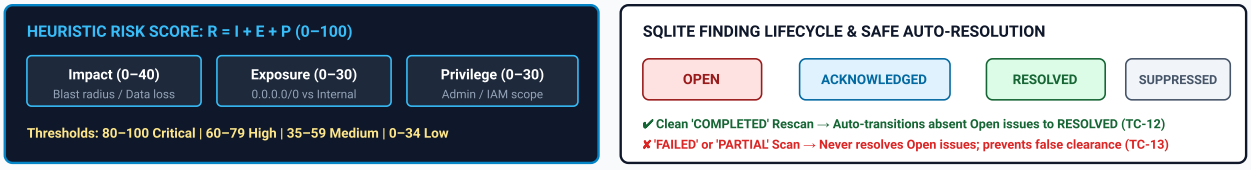


Figure 4.1: Heuristic Scoring Breakdown (`risk_scoring.py`) and Stateful Lifecycle Rules (`repository.py`)

- **`calculate_heuristic_risk(impact, exposure, privilege)` (`risk_scoring.py`):** Clamps each input to its valid range (`0..40`, `0..30`, `0..30`), sums them into `total_score` (`0..100`), and maps to `Critical` (≥80), `High` (≥60), `Medium` (≥35), or `Low` (<35). If any factor is `None`, it returns `total_score=None, severity="Unknown"`.
- **`compute_finding_fingerprint(rule_id, resource_id, evidence)` (`deduplication.py`):** Strips volatile metadata keys (`"source"`, `"collected_at"`) from the evidence dictionary, serializes `{"rule_id", "resource_id", "evidence"}` into canonical sorted JSON (`sort_keys=True, separators=(",", ":")`), hashes it with `hashlib.sha256`, and returns `f"fp-{digest[:24]}"`.
- **`run_resource_scan(bundle, rules, include_events, ...)` (`scanner.py`):** Orchestrates both resource rules and CloudTrail event rules. Explicitly sets `ScanExecutionResult.status` to `"failed"` (if 0 items and >0 diagnostics), `"partial"` (if items scanned alongside >0 diagnostics), or `"completed"` (if 0 diagnostics).

07. SQLite Persistence & Report Engine (`db/` & `reports/`)

`init_db()` (`models.py`) creates four tables (`scans`, `findings`, `events`, `audit_log`) and executes `os.chmod(path, 0o600)` so only the file owner on Kali Linux can read or modify the database. Below are the four core database and reporting workflows:

1. `save_scan_and_sync_findings()`
Runs inside an atomic SQLite transaction. Inserts the scan row, upserts findings by `fingerprint` (updating `last_seen` on repeat sightings or re-opening `Resolved` issues if they recur), and auto-resolves vanished findings **only** when `scan.status == 'completed'`.

2. `update_finding_status()`
Validates target status against `ALLOWED_FINDING_STATUSES`. Strictly raises a `ValueError` if an analyst selects `Suppressed` with an empty justification string, and appends an immutable JSON record to the `audit_log` table.

3. `save_events()` & `list_findings()`
`save_events()` scrubs secrets via `redact_sensitive_fields()` before inserting into `events`. `list_findings()` supports parameterized SQL filtering by status, severity, rule family (`NET-%`), scan ID, and search substring.

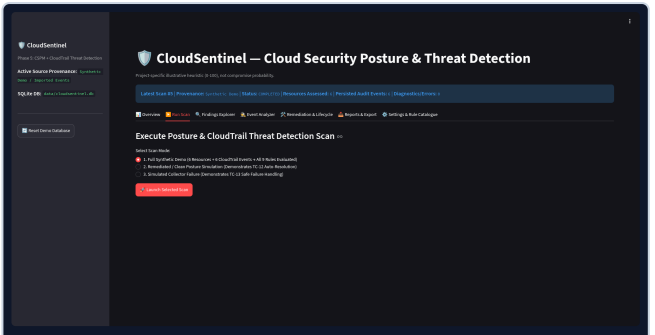
4. `generate_json_report()` & `CSV`
Located in `report_generator.py`. Exports selected scan findings with embedded `generated_at_utc`, `provenance_label` (`Synthetic Demo`, `Imported Events`, or `AWS Read-only Scan`), and heuristic disclaimers (`TC-14`).

08. Visual Walkthrough: The 7-Tab Streamlit Analyst Workbench (`app.py`)

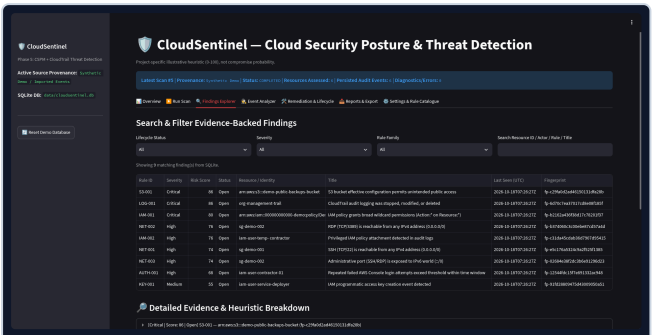
Below are the 8 verified screenshots captured directly from the running CloudSentinel application on Kali Linux, illustrating how `app.py` connects every backend service into a unified analyst workflow.



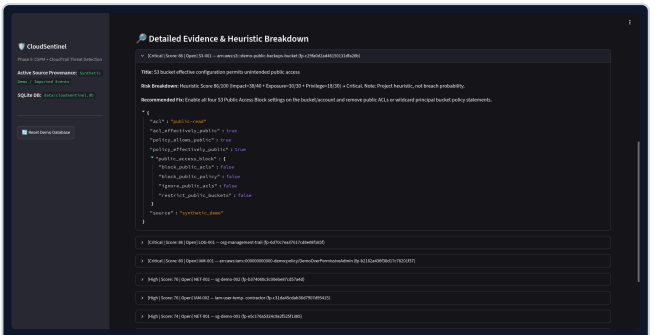
1. Overview Tab: 9 Tracked Findings, Provenance Banner & Severity Charts



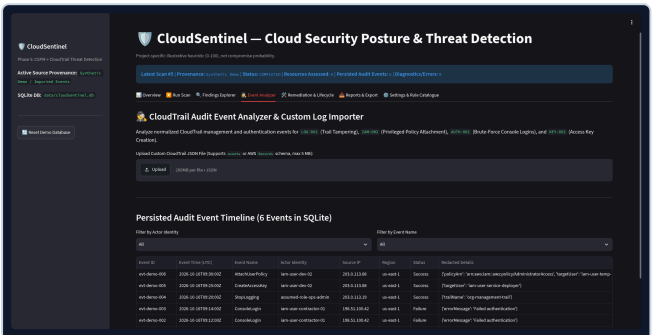
2. Run Scan Tab: Synthetic Demo, Clean Remediation (TC-12) & Error Sim (TC-13)



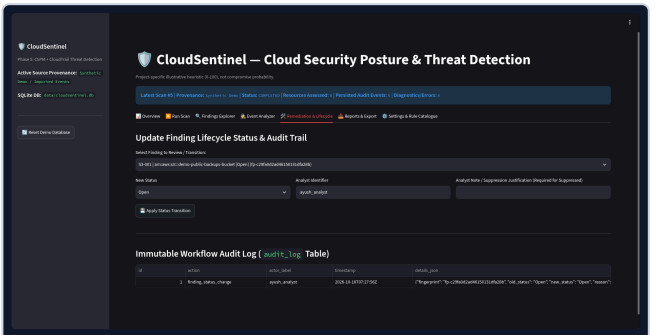
3. Findings Explorer: Multi-Criteria Filters (Status, Severity, Family) & Fingerprints



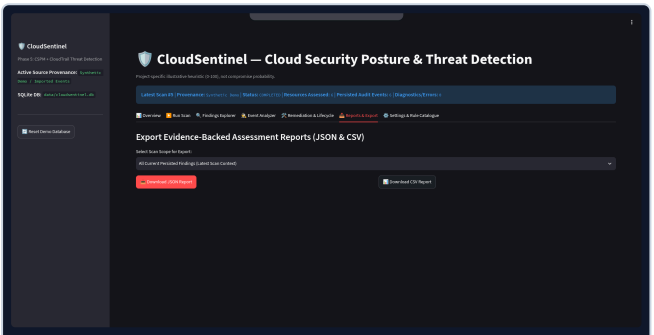
4. Evidence Inspector: S3-001 Heuristic Breakdown (86/100) & Raw JSON Evidence



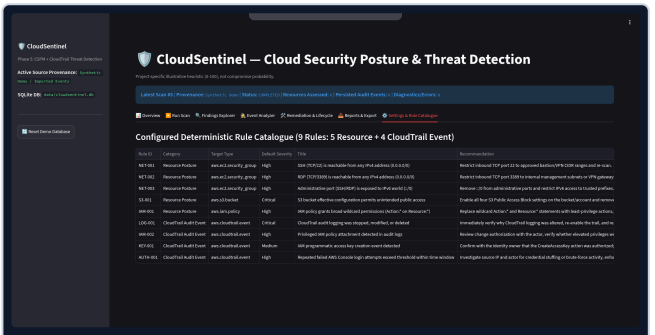
5. CloudTrail Event Analyzer: Custom JSON Uploader & Normalized Timeline



6. Remediation & Lifecycle: Status Transitions & Immutable Audit Log Table



7. Reports & Export: 1-Click Provenance-Labeled JSON & CSV Report Downloads



8. Rule Catalogue: Reference View of All 9 Resource & CloudTrail Detectors

09. Automated Verification Matrix (`TC-01` to `TC-14` — 35 Passing Tests)

Test IDs	Test File (`tests/`)	Verified Security & Functional Behavior	Result
`TC-01..03`	`test_network_rules.py`	Detects public SSH (22), RDP (3389), IPv6 (`::/0`); ignores private CIDRs; flags missing permissions as diagnostics.	**5 PASSED**
`TC-04`	`test_s3_rules.py`	Detects effective public S3 buckets; verifies PublicAccessBlock overrides public ACLs safely; flags missing PAB.	**3 PASSED**
`TC-05`	`test_iam_rules.py`	Detects unconstrained `Action:*` on `Resource:*` ; ignores scoped least-privilege S3 policies.	**2 PASSED**
`TC-06..08`	`test_event_rules.py`	Sliding-window brute force below/above threshold, routine events ignored, `LOG-001` / `IAM-002` / `KEY-001` , secret redaction.	**5 PASSED**
`TC-09`	`test_phase1_foundation.py` & `test_security_hardening.py`	Corrupt JSON syntax, missing required fields, and oversized (>5 MB) payloads rejected safely without crash.	**6 PASSED**
`TC-10`	`test_aws_provider.py`	Mocked Boto3 `AccessDenied` marks scan `failed` or `partial` —never falsely reports a clean scan.	**2 PASSED**
`TC-11..13`	`test_persistence_and_lifecycle.py`	Repeat scan deduplication, clean rescan auto-resolution, failed scan protection, mandatory suppression reason + audit log.	**4 PASSED**
`TC-14` & `FR-05`	`test_reports_and_filtering.py` & `test_risk_scoring_and_scanner.py`	JSON/CSV exports embed provenance & UTC timestamps; multi-criteria SQL filtering; deterministic SHA-256 hashes; 9-rule run.	**6 PASSED**
Hardening	`test_security_hardening.py`	Verifies SQLite database enforces `0o600` owner-only permissions and scans entire repo for hardcoded `AKIA/ASIA` keys.	**2 PASSED**

10. Top 7 Viva & Cybersecurity Interview Questions & Answers

1. **Q: Why did you use deterministic rules instead of Machine Learning / AI for threat detection?**
   Ans: In cloud posture auditing, rules like "Port 22 open to 0.0.0.0/0" or "CloudTrail StopLogging called" have well-defined specifications. Deterministic rules guarantee 100% explainability, zero hallucination, and reproducible evidence—whereas black-box ML models cannot easily justify why a configuration violates compliance.
2. **Q: How does CloudSentinel prevent duplicate alerts when scanning the same account repeatedly (`TC-11`)?**
   Ans: `compute_finding_fingerprint()` generates a canonical SHA-256 digest from `rule_id + resource_id + sorted(evidence)` (excluding timestamps). The SQLite `findings.fingerprint` column has a `UNIQUE` constraint; repeat scans simply update `last_seen` and `scan_id` .
3. **Q: What happens if an AWS scan fails halfway due to `AccessDenied`? Will existing open findings get marked as Resolved (`TC-10` & `TC-13`)?**
   Ans: Never. `aws_provider.py` catches `ClientError` and records a diagnostic, which sets the scan status to `partial` or `failed` . In `repository.py` , auto-resolution only executes if `scan_result.status == "completed"` .
4. **Q: How does your Risk Score work? Is a score of 86/100 a 86% probability of being hacked?**
   Ans: No. We explicitly label the score as a *prioritization heuristic*: `R = Impact(0–40) + Exposure(0–30) + Privilege(0–30)` . It ranks which misconfiguration an analyst should fix first based on blast radius, internet reachability, and IAM privilege.